代码调试面试题

车间生产看板系统 · Debug & 需求分析

线下独立完成版 | AI 工具辅助编程能力评估

完成时间	请在规定时间内独立完成，完成后提交 Git 仓库链接，不计时（建议 40 分钟）
允许工具	可使用 ChatGPT / Claude / Cursor / Copilot 等任何 AI 工具，鼓励使用
提交方式	将代码提交至 Git 仓库（GitHub），并将仓库链接发送给招聘联系人
后续流程	提交后安排线上视频复盘（约 30 分钟），与面试官共同回顾你的解题过程
评估重点	需求分析能力 · AI 工具调用质量 · Bug 定位与修复 · Git 规范使用 · 代码理解与表达

一、背景与系统介绍

你接到了一个紧急求助：

> 车间 IT 同事写了一套「生产工单看板系统」，用于显示当日各工单的计划数量、完成进度和状态。
>
> 这套系统昨天还能用，今天却出现了多个问题。他临时请假，你需要接手并修复它，让车间主任能正常使用。
>
> **补充知识**：所谓“生产工单”，是工厂内部用于组织和控制生产的任务单，相当于一张制造指令卡，明确规定“生产什么产品、生产多少、由哪个班组或设备完成、何时完成”。车间的实际生产活动都围绕工单展开，报工数据也以工单为单位进行统计。

系统结构

- 后端：Flask 提供 REST API，数据存储在 SQLite 数据库（文件：workshop.db）
- 前端：一个嵌入在 Python 文件中的 HTML 页面，通过 fetch() 调用后端接口
- 运行方式：python app.py，然后浏览器打开 http://localhost:5000
- 依赖安装：pip install flask

API 接口说明

系统包含以下 REST API，修复后应全部正常工作：

方法 + 路径	功能	说明 / 示例
GET /api/orders	获取所有工单	支持 ?status=pending / in_progress / completed 筛选
GET /api/orders/{id}	获取单个工单	按 ID 查询，找不到返回 404
PUT /api/orders/{id}/progress	更新完成数量	请求体（JSON）：{"completed": 150}，返回更新后的完整工单对象

GET /api/summary	获取汇总数据	返回总计划件数、总完成件数、整体完成率、各状态工单数

工单状态定义

状态值	中文名称	触发条件
pending	待生产	completed = 0
in_progress	生产中	0 < completed < quantity
completed	已完成	completed >= quantity

车间主任需求描述（原话记录）

> 我要能在大屏幕上看到今天每张工单的情况：计划做多少、已经做了多少、完成百分比是多少。工单分三个状态：没开始（待生产）、做一半（生产中）、做完了（已完成）。右边要有个输入框，我输入已完成数量，一点更新，数字和状态就自动变。最上面要有个汇总——今天总共计划多少件、完成了多少件、整体完成率是多少。

注：「整体完成率」的计算方式为：总完成件数 ÷ 总计划件数 × 100%（以件数为分母，不是以工单条数为分母）

已知问题报告（IT 同事的便条）

- 系统启动后，筛选特定状态的工单会报错
- 点击"更新"按钮后看起来成功了，但刷新页面数据没变
- 进度条不显示
- 汇总里的"整体完成率"数字感觉不对
- 其他可能还有一些隐藏的问题，具体不确定

二、你的任务

任务 A：代码调试与修复（主体任务）

打开 app.py，运行它，找到并修复所有导致功能异常的 Bug。

注意事项：
* 大多数 Bug 的修改只需 1–2 行代码即可解决。
* 不要重构结构，不要大面积重写代码。
* 不要新增复杂封装、抽象层或改动数据库结构
* 不要修改 workshop.db 中的测试数据
*仅在现有结构上做“最小必要修改”

修复完成的判断标准：

- GET /api/orders?status=in_progress 能正常返回数据（不报错）
- GET /api/summary 返回的 overall_progress 数值正确
- PUT /api/orders/{id}/progress 更新后刷新页面数据持久化
- 浏览器页面打开后进度条可见，各状态彩色标签正确显示

建议：用 AI 工具辅助调试时，请保留你与 AI 的对话记录（截图或导出），
复盘时面试官会请你展示并讲解你的提问方式和 AI 使用过程。

任务 B：Git 提交规范（必须完成）

α **本环节考察你对 Git 的日常使用习惯，请认真对待。**
面试官会直接查看你的 commit 记录和仓库结构。

请按照以下步骤完成 Git 操作：

1. **初始化仓库**

- 在本地新建 Git 仓库，并将 app.py 和 workshop.db 作为初始文件提交（初始 commit）
- 在 GitHub 上创建远程仓库并推送

2. **按 Bug 分批提交**

- 每修复 1–3 个相关 Bug，提交一次 commit（不要把所有修改放在同一个 commit 里）
- commit message 请用中文或英文简要描述你修复了什么，例如：

```
fix: 修复 SQL 参数化查询，防止 status 筛选报错
fix: 修复 conn.commit() 缺失导致进度更新不持久化
fix: 修复 JS 模板字符串语法错误，进度条现在可正常显示
```

3. **添加 README（可选加分）**

- 在仓库根目录创建 README.md，说明如何运行这个系统（依赖安装、启动命令等）

4. **推送并发送链接**

- 将所有 commit push 到远程仓库，并将仓库链接发送给招聘联系人
- 请确认仓库为 Public（公开）或已邀请面试官为协作者

三、附件说明

本题所需文件已单独发送，包含：

文件名	说明
app.py	包含 Bug 的完整系统代码（后端 + 嵌入式前端），请直接在此文件上修改
workshop.db	SQLite 数据库文件，包含测试工单数据，与 app.py 放在同一目录下即可

注意事项：

1. 请不要修改 workshop.db 中的测试数据，只修复代码逻辑
2. 修复过程请保留 AI 工具对话记录，复盘时需要展示
3. 如遇环境配置问题（Python 版本、依赖安装等），可先自行解决，若无法解决请在提交时说明
4. 仓库请设置为 Public 或提前邀请面试官

祝你发挥顺利，期待复盘时的交流！